Module Code	IY4113
Group	A
Module Title	Practical assignment part 1: Java Fundamentals
Assessment Type	Report
Module Tutor Name	Jonathan Shore
Student ID Number	P488018
Assessment Window	Final submission: 16/02/2026- 22/02/2026

✓ *I confirm that this assignment is my own work. Where I have referred to academic sources, I have provided in-text citations and included the sources in the final reference list.*

✓ *Where I have used AI, I have cited and referenced appropriately.*

KAPLAN INTERNATIONAL PATHWAYS

Table of Contents

INTRODUCTION 3

ANALYSIS 3

ALGORITHM DESIGNS 7

TESTING 15

EVALUATION 16

CODE LISTING 17

REFERENCES 31

APPENDICES 32

Introduction

This project involved designing and making a console-based Java application called CityRide Lite which is a daily travel tracker for public transport users. The program allows a passenger to record details of journeys made during a single day including travel zones, date, time band (peak or off-peak) and their passenger type. Using this information, the system calculates the correct fare by referencing a predefined dataset, applies any relevant discounts, and ensures that the total cost does not exceed the daily spending cap for that passenger category.

The application operates through a menu-driven interface that runs continuously until the user chooses to exit. When in use the user can add journeys, view all recorded journeys, filter them by a specific criteria, remove incorrect entries or reset the entire day's data. The system also generates summaries showing total journeys, total cost, average cost per journey and the most expensive trip.

All data is stored temporarily in memory and is not saved once the program closes. The project shows great programming skills such as input validation, control structures, collections and object-oriented design. At last, CityRide Lite creates a simple but realistic simulation of a transport fare system while meeting the functional requirements specified in the assessment.

Analysis

The CityRide Lite system is created to manage transport journeys for a single passenger in a day. Its main function is to calculate accurate fares based on journey details while allowing the user to organise and review their travel information. To make this happen the system collects user input, validates it, performs calculations using fixed rules and stores the results for later use within the same session.

System Requirements

The program must provide a clear menu that allows the user to perform essential tasks. The most important feature is adding a journey, which requires entering the date, starting zone, destination zone, time band, and passenger type. The system calculates the number of zones crossed using a formula and determines the base fare from the dataset. It then applies the correct discount and checks whether the daily cap has been reached.

Additional required functions include:

Listing all recorded journeys

Filtering journeys by passenger type, time band, zone or date

Removing a journey using its ID

Resetting the session data after confirmation

Displaying summaries and totals

Maintaining running totals per passenger type

Continuing operation until the user exits

Constraints

The program operates under some limitations. It is a console application with no graphical interface and does not store data permanently. Valid travel zones are limited to values between 1 and 5. Passenger types and time bands are restricted to specific predefined options. Fare calculations must use the provided dataset without modification.

The system must also handle invalid input safely by displaying error messages and requesting new input rather than terminating. Monetary values must be rounded to two decimal places and daily caps must be enforced separately for each passenger type.

Overall, the system must be accurate, reliable and user-friendly while strictly following the rules provided in the specification

Input, Process and Output

Startup & Session Setup

Input	Process	Output
Passenger name (optional)	Store name in session (if provided)	Name stored for display
Passenger type selection (1–4)	Validate input range	Selected passenger type stored
Invalid input	Display error message and re-prompt	Correct value obtained

Add Journey

Input	Process	Output
Date (non-blank string)	Validate not empty	Valid date stored
From zone (1–5)	Validate numeric and range	Zone accepted
To zone (1–5)	Validate numeric and range	Zone accepted
Time band (Peak/Off-peak)	Validate allowed values	Time band stored
Passenger session type	Retrieve discount rate	Discount determined
Zones crossed	Calculate abs(to − from) + 1	Number of zones
Base fare	Lookup from dataset	Base fare value
Discount	Apply passenger discount	Discounted fare
Running total	Compare with daily cap	Final charged fare

List All Journeys

Input	Process	Output
User selects “List Journeys”	Check if list is empty	If empty → message shown
Stored journeys	Iterate through list	Each journey displayed
Journey details	Format fields for display	Structured output

Filter Journeys

Input	Process	Output
Filter type selection	Validate option	Correct filter applied
Passenger type / band / zone / date	Compare against stored journeys	Matching journeys identified
Matching records	Collect results	Filtered list displayed
No matches	Detect empty result set	“No journeys found” message

Daily Summary

Input	Process	Output
Stored journeys	Group by passenger type	Separate totals calculated
Base fares	Sum before discount	Pre-discount total
Discounted fares	Sum after discount	Discounted total
Charged fares	Sum after cap	Final charged total
Running totals	Compare with cap	Cap reached status

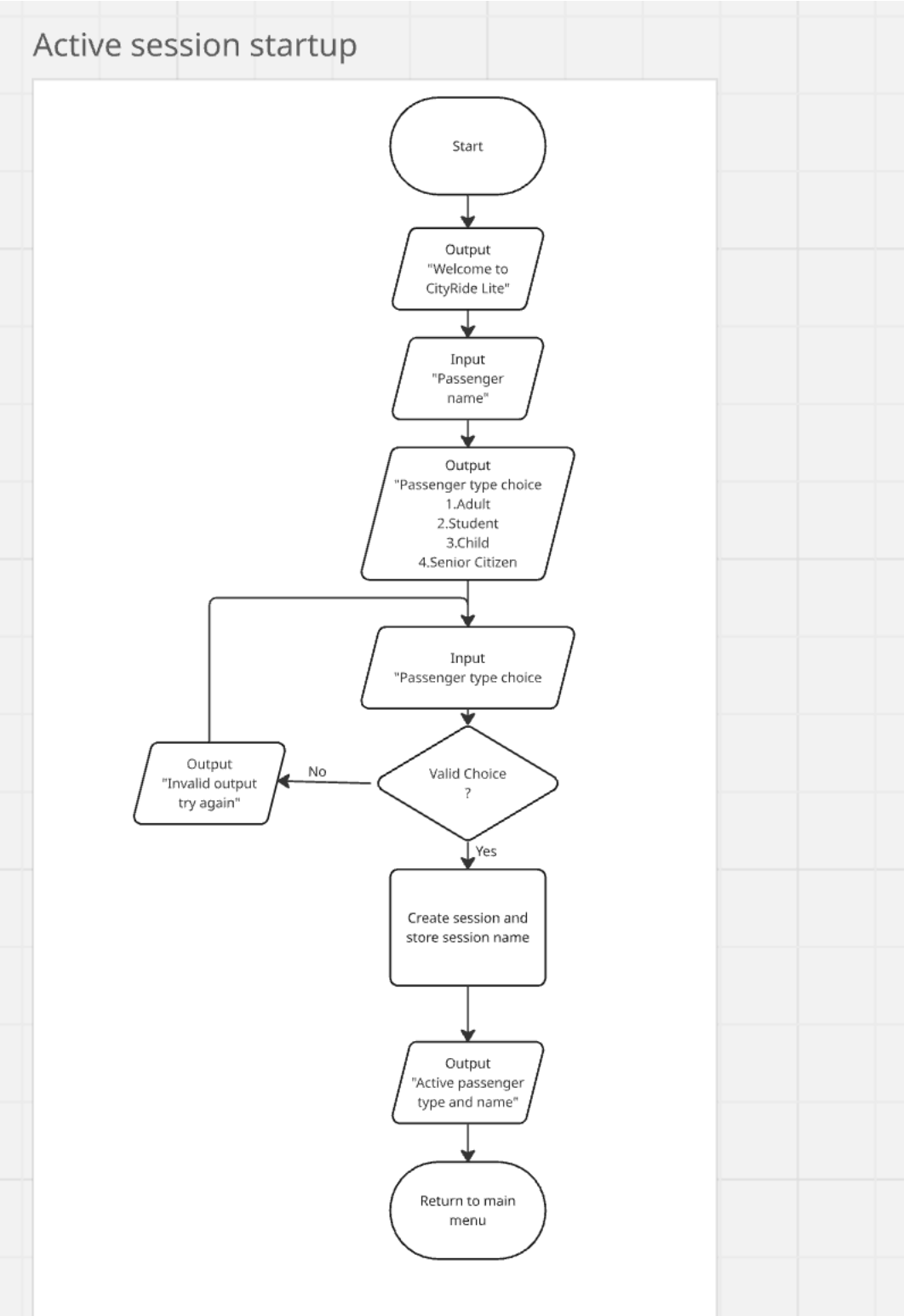
Totals by Passenger Type

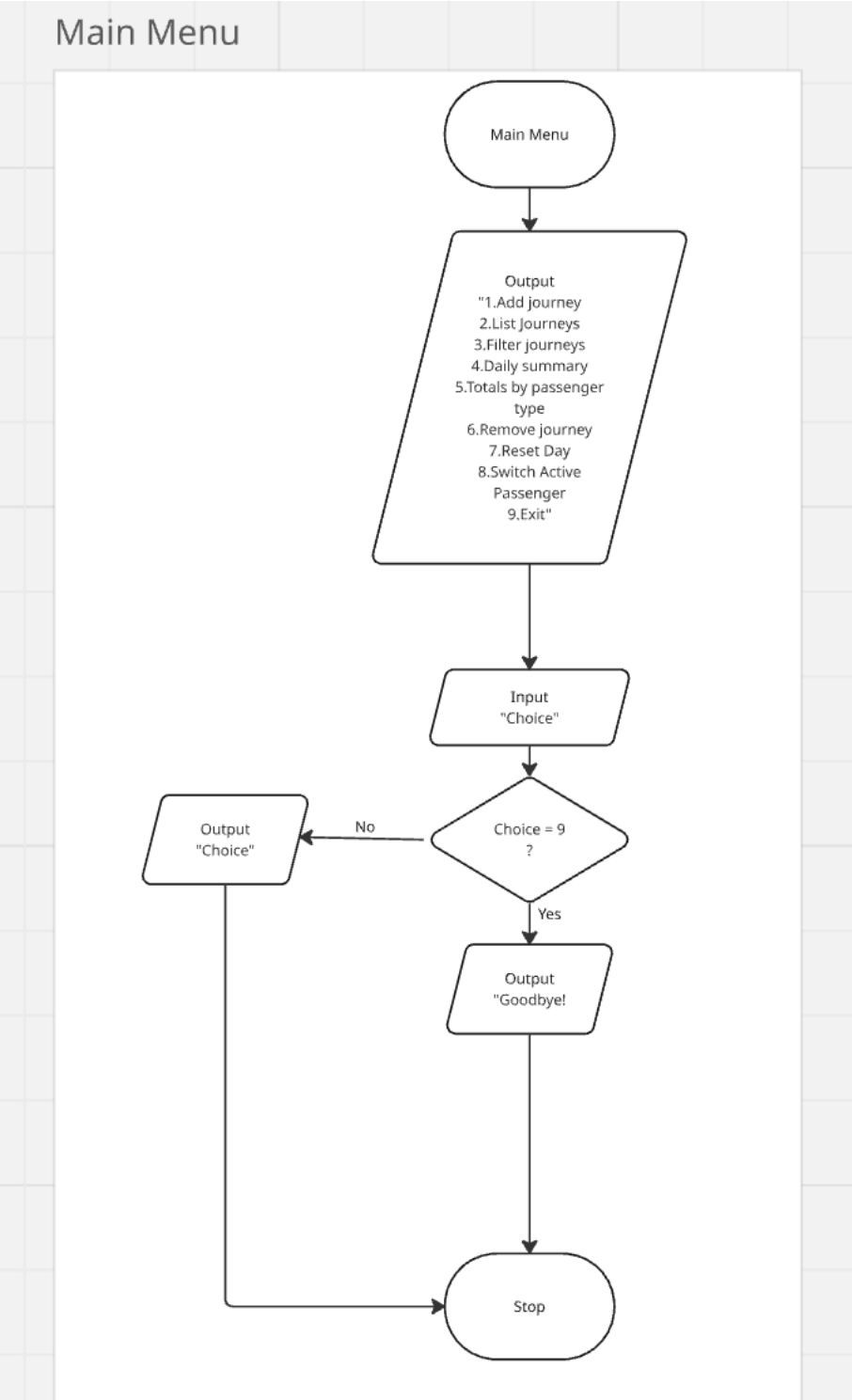
Input	Process	Output
Journey ID	Validate numeric input	Valid ID accepted
ID exists	Check list	If not → error message
Confirmation (yes/no)	Validate response	Proceed or cancel
Confirmed removal	Delete journey	Journey removed
Remaining journeys	Recalculate totals and caps	Updated values displayed

Reset Day

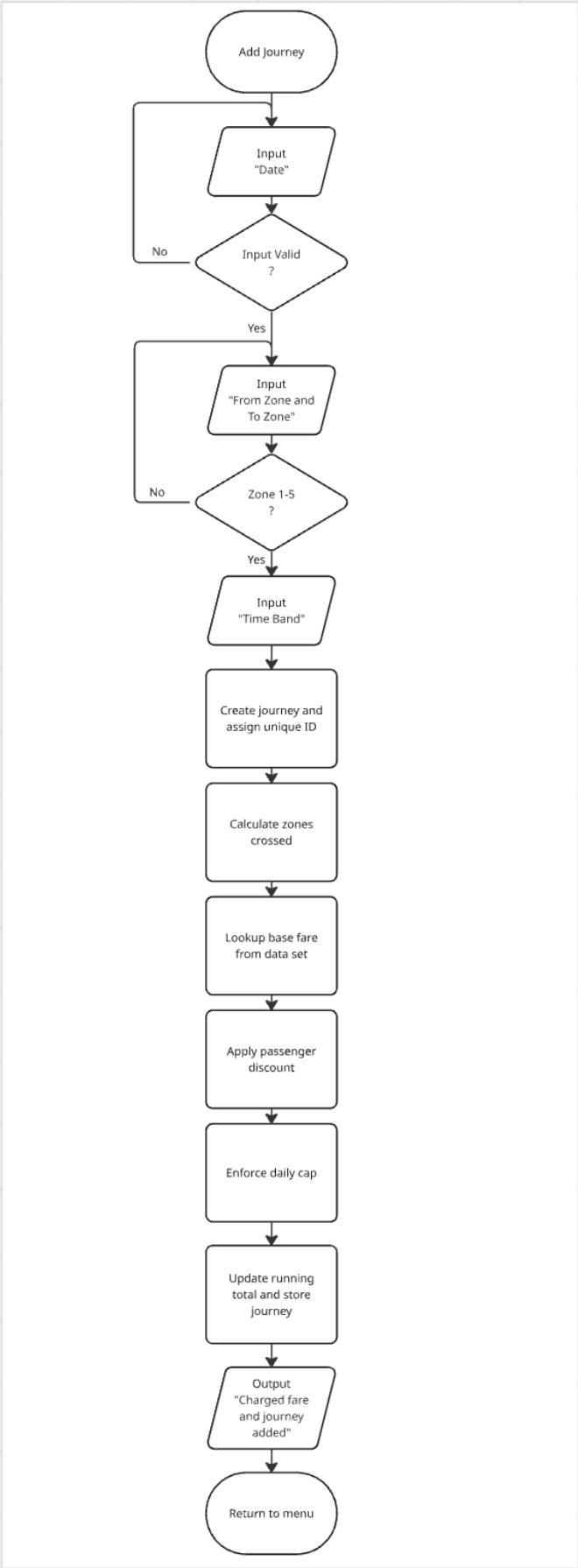
Input	Process	Output
Confirmation (yes/no)	Validate response	Continue or cancel
Yes selected	Clear journeys list	All journeys removed

Algorithm designs

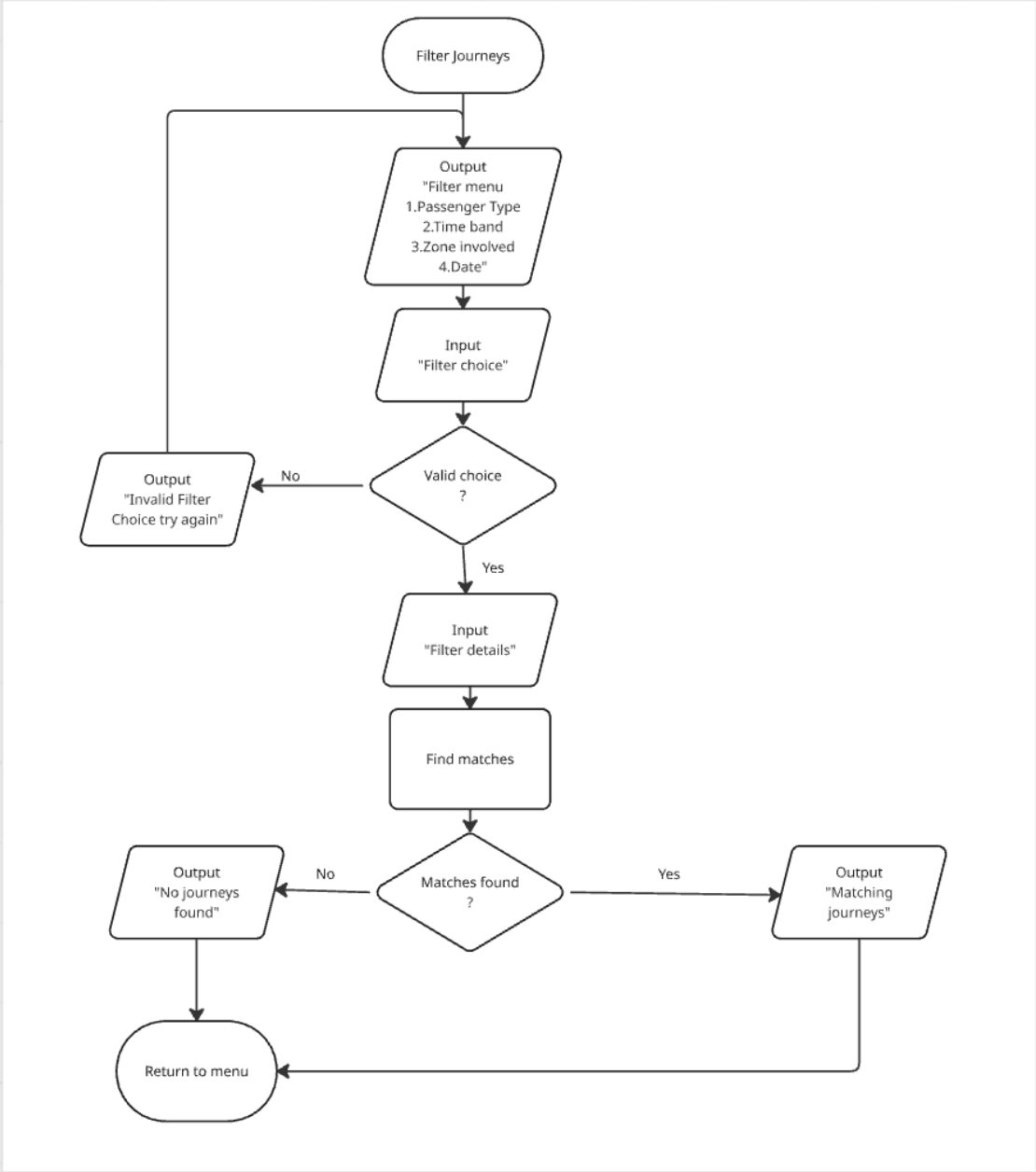



Main Menu
Main Menu
Output
"1.Add journey
2.List Journeys
3.Filter journeys
4.Daily summary
5.Totals by passenger type
6.Remove journey
7.Reset Day
8.Switch Active Passenger
9.Exit"
Input
"Choice"
Choice = 9 ?
No
Output
"Choice"
Yes
Output
"Goodbye!
Stop

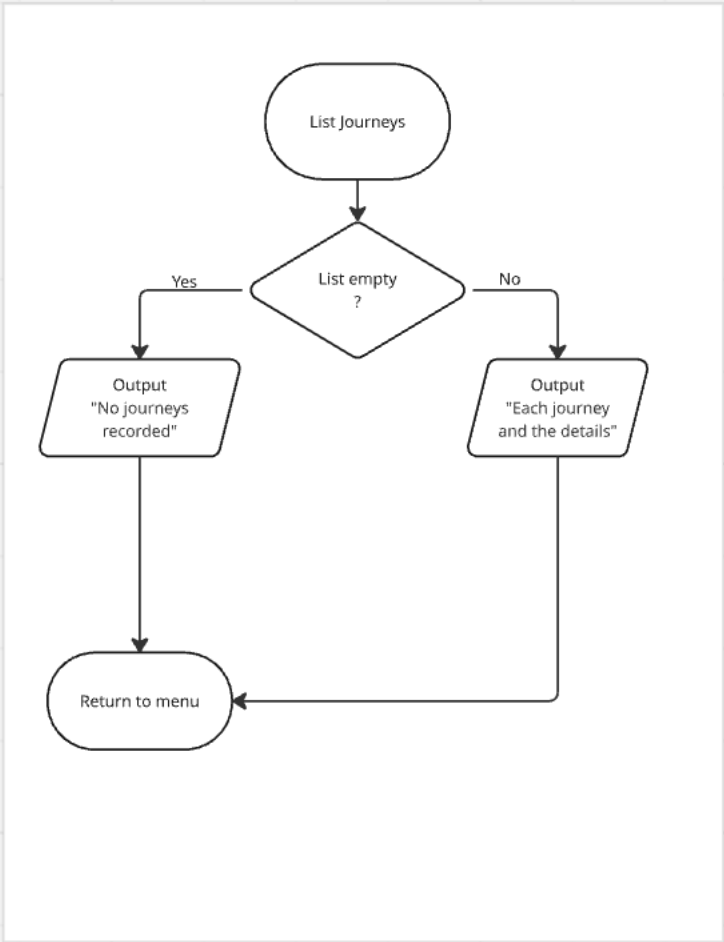
Add Journey


Add Journey
Input "Date"
Input Valid ?
No
Yes
Input "From Zone and To Zone"
Zone 1-5 ?
No
Yes
Input "Time Band"
Create journey and assign unique ID
Calculate zones crossed
Lookup base fare from data set
Apply passenger discount
Enforce daily cap
Update running total and store journey
Output "Charged fare and journey added"
Return to menu

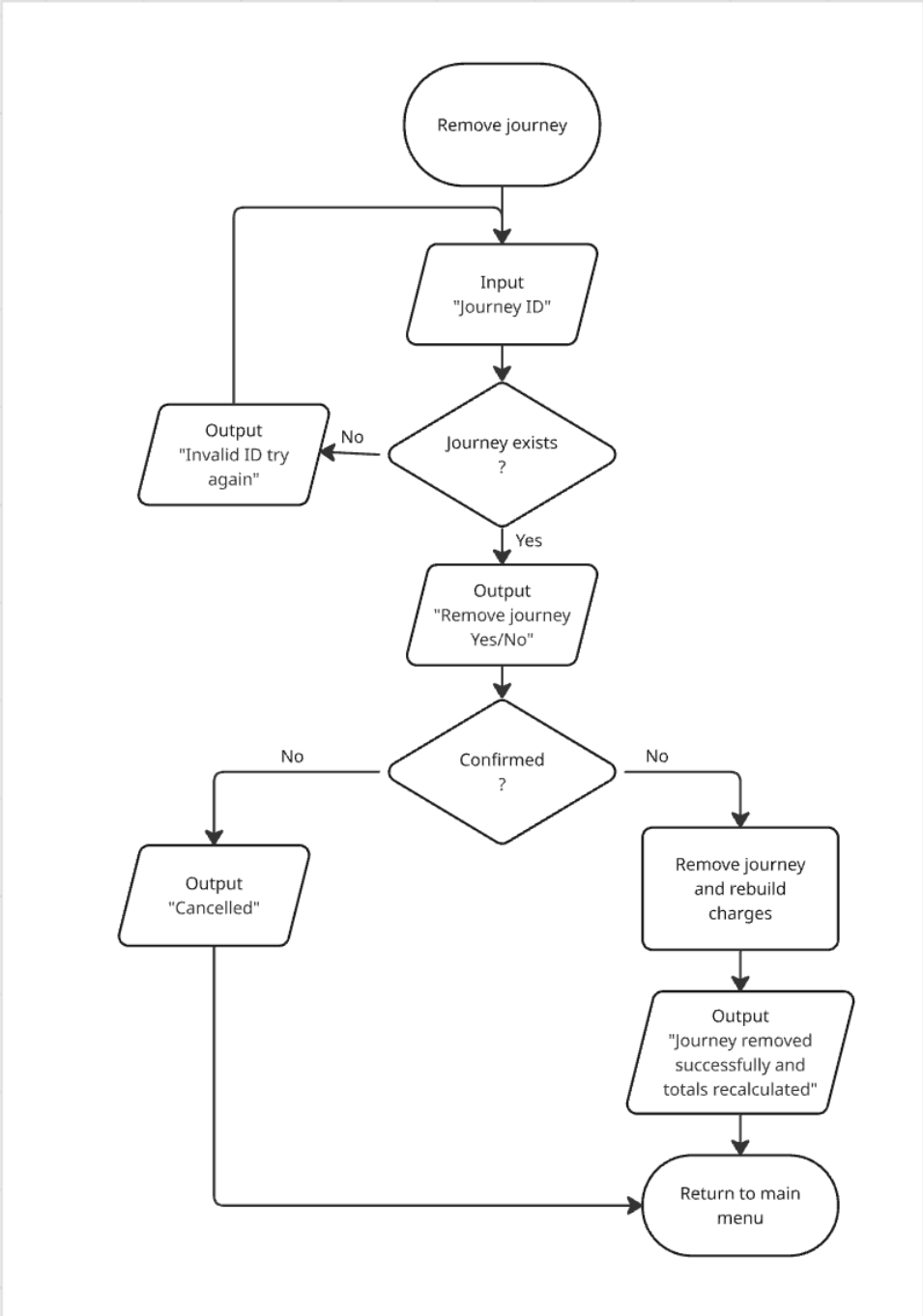
Filter journeys

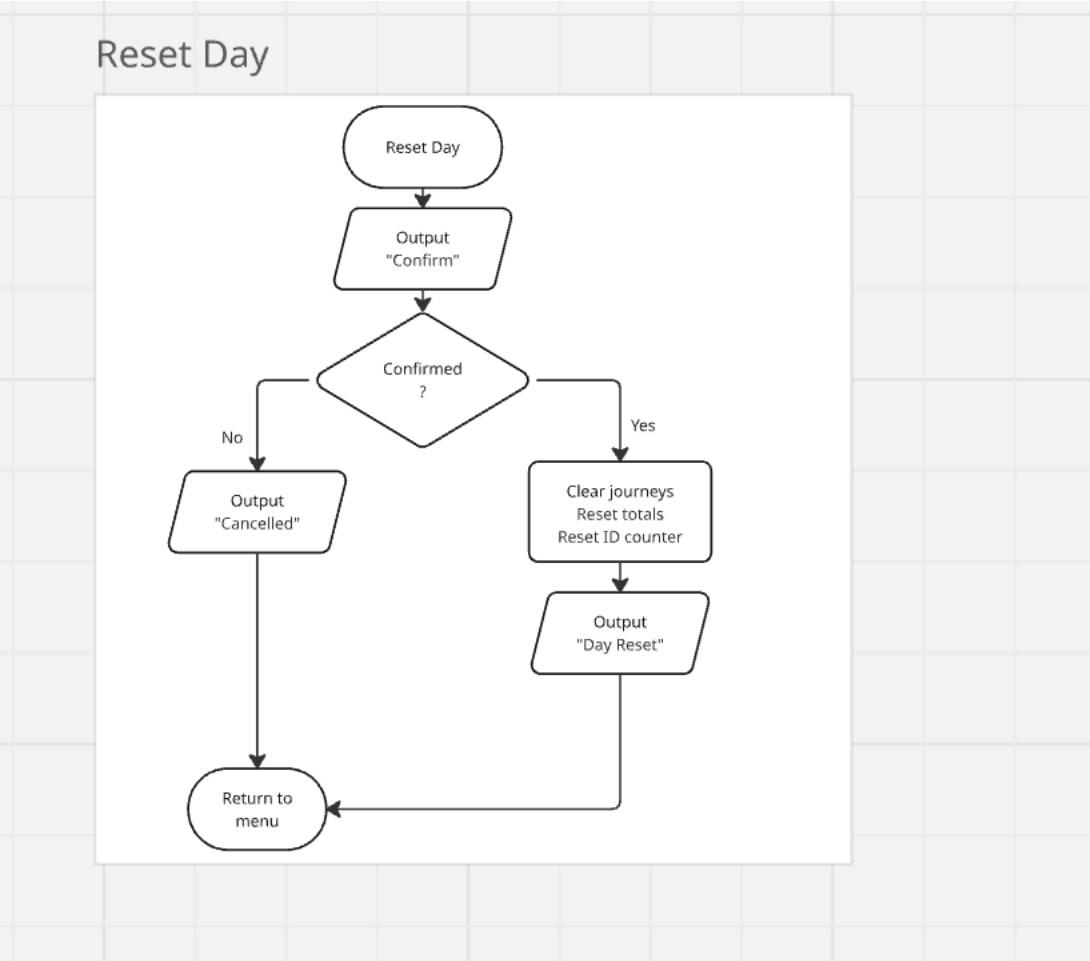


List journeys

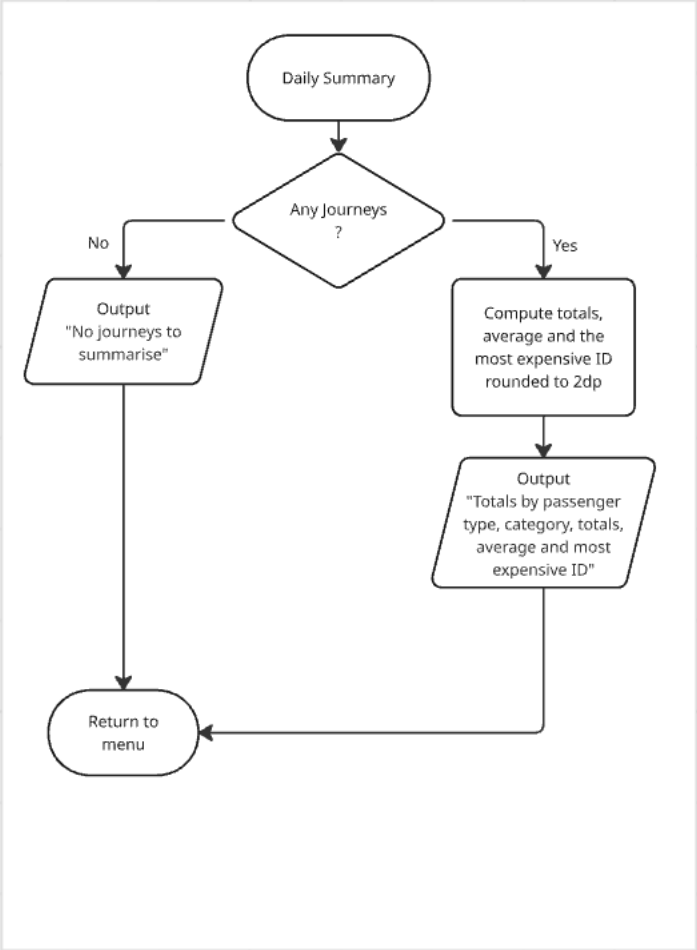


Remove journey




Reset Day
Reset Day
Output "Confirm"
Confirmed ?
No
Yes
Output "Cancelled"
Clear journeys
Reset totals
Reset ID counter
Output "Day Reset"
Return to menu

Daily Summary



Testing

Test Number	Test	Test Description	Test Data	Expected Outcome	Actual Outcome	Comments
1	Item	Error test	press Enter at Date prompt– empty string	Program should respond with an error message – input cannot be empty	Input required.	Test passes
2	Amount	Invalid	0 – integer (zone)	Program should respond with an error message – value out of range	Zone must be 1-5	Test passes
3	Amount	Extreme valid test	1 – integer (zone)	Program should continue as expected	Zone accepted and program continues to next prompt	Test passes
4	Amount	Extreme valid test	5 – integer (zone)	Program should continue as expected	Zone accepted and program continues to next prompt	Test passes
5	Amount	Invalid extreme test – range test	6 – integer (zone)	Program should respond with an error message – value out of range	Zone must be 1-5	Test passes
6	Amount	Error test	abc – string (zone/menu input)	Program should respond with an error message – invalid number	Invalid number	Test passes
7	Item	Valid test	peak – string (time band)	Program should continue as expected	Time band accepted and program continues	Test passes
8	Item	Valid test	off-peak – string (time band)	Program should continue as expected	Time band accepted and program continues	Test passes
9	Item	Invalid	morning – string (time band)	Program should respond with an error message – incorrect value	Enter Peak or Off-peak	Test passes
10	Amount	Invalid	0 – integer (passenger menu)	Program should respond with an error message – incorrect value	Choose 1-4	Test passes
11	Item	Error test	maybe – string (yes/no)	Program should respond with an error message – incorrect value	Type yes or no	Test passes
12	Item	Error test	List journeys when none recorded	Program should respond with an error message – no data available	No journeys recorded	Test passes
13	Item	Error test	Daily summary when none recorded	Program should respond with an error message – no data available	No journeys to summarise	Test passes
14	Item	Invalid	Filter menu choice 9 – integer	Program should respond with an error message – incorrect value	Invalid filter choice	Test passes
15	Item	Valid test	Remove journey with ID not present	Program should respond with an error message – ID not found	No journey with ID	Test passes
16	Item	Valid test	Remove journey confirmation = no	Program should cancel operation	Cancelled	Test passes
17	Item	Valid test	Reset day confirmation = no	Program should cancel operation	Cancelled	Test passes
18	Item	Valid test	Reset day confirmation = yes	Program should clear stored journeys	Day reset	Test passes
19	Item	Valid test	Exit program	Choose option 9	Program should terminate	Goodbye!
20	Item	Invalid	Main menu choice 99 – integer	Program should respond with an error message – incorrect choice	Invalid choice	Test passes

Evaluation

The CityRide Lite application successfully meets most of the functional requirements outlined in the specification. The program accurately calculates fares, applies discounts, enforces daily caps and stores journeys in memory. The menu-driven interface is straightforward to use allowing users to perform all required actions without difficulty.

A major strength of the system is its robust input validation which prevents incorrect data from affecting calculations. The ability to filter journeys and generate summaries provides useful insights into travel behaviour. Additionally, maintaining separate totals for each passenger type ensures that caps are applied correctly.

However, the program also has some limitations. As a console application, it lacks a graphical interface which may reduce usability for some users. Data is not saved permanently meaning all information is lost once the program closes. The system also supports only a single passenger session at a time and does not allow editing of existing journeys.

Future improvements could include adding file storage, a graphical user interface, support for multiple passengers and more advanced reporting features. Despite these limitations the program fulfills its intended purpose and demonstrates a solid understanding of object-oriented programming principles and software design.

Code Listing

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Scanner;

//
==========================================================================
==
//  CityRide Lite
//  - Active passenger session (type + optional name)
//  - Add / list / filter / daily summary
//  - Totals by passenger type (pre-discount, post-discount, charged, cap
reached)
//  - Category counts (peak/off-peak + zone involvement + route pair
counts)
//  - Remove journey recalculates charges properly
//
==========================================================================
==


//
==========================================================================
==
//  CityRideDataset
//
==========================================================================
==
final class CityRideDataset {
    private CityRideDataset() {}

    public static final int MIN_ZONE = 1;
    public static final int MAX_ZONE = 5;

    public enum TimeBand { PEAK, OFF_PEAK }
    public enum PassengerType { ADULT, STUDENT, CHILD, SENIOR_CITIZEN }

    public static final Map<PassengerType, BigDecimal> DISCOUNT_RATE =
Map.of(
            PassengerType.ADULT,          new BigDecimal("0.00"),
            PassengerType.STUDENT,        new BigDecimal("0.25"),
            PassengerType.CHILD,          new BigDecimal("0.50"),
            PassengerType.SENIOR_CITIZEN, new BigDecimal("0.30")
    );
```

```
    public static final Map<PassengerType, BigDecimal> DAILY_CAP = Map.of(
            PassengerType.ADULT,          new BigDecimal("8.00"),
            PassengerType.STUDENT,        new BigDecimal("6.00"),
            PassengerType.CHILD,          new BigDecimal("4.00"),
            PassengerType.SENIOR_CITIZEN, new BigDecimal("7.00")
    );

    public static final Map<String, BigDecimal> BASE_FARE =
buildBaseFare();

    public static BigDecimal getBaseFare(int from, int to, TimeBand band) {
        return BASE_FARE.get(key(from, to, band));
    }

    public static String key(int f, int t, TimeBand b) { return f + "-" + t
+ "-" + b.name(); }

    private static BigDecimal money(String a) { return new
BigDecimal(a).setScale(2, RoundingMode.HALF_UP); }

    private static Map<String, BigDecimal> buildBaseFare() {
        Map<String, BigDecimal> m = new HashMap<>();

        // Peak
        put(m,1,1,TimeBand.PEAK,"2.50"); put(m,1,2,TimeBand.PEAK,"3.20");
put(m,1,3,TimeBand.PEAK,"3.80"); put(m,1,4,TimeBand.PEAK,"4.40");
put(m,1,5,TimeBand.PEAK,"5.00");
        put(m,2,1,TimeBand.PEAK,"3.20"); put(m,2,2,TimeBand.PEAK,"2.30");
put(m,2,3,TimeBand.PEAK,"3.10"); put(m,2,4,TimeBand.PEAK,"3.80");
put(m,2,5,TimeBand.PEAK,"4.50");
        put(m,3,1,TimeBand.PEAK,"3.80"); put(m,3,2,TimeBand.PEAK,"3.10");
put(m,3,3,TimeBand.PEAK,"2.10"); put(m,3,4,TimeBand.PEAK,"3.00");
put(m,3,5,TimeBand.PEAK,"3.70");
        put(m,4,1,TimeBand.PEAK,"4.40"); put(m,4,2,TimeBand.PEAK,"3.80");
put(m,4,3,TimeBand.PEAK,"3.00"); put(m,4,4,TimeBand.PEAK,"2.00");
put(m,4,5,TimeBand.PEAK,"2.90");
        put(m,5,1,TimeBand.PEAK,"5.00"); put(m,5,2,TimeBand.PEAK,"4.50");
put(m,5,3,TimeBand.PEAK,"3.70"); put(m,5,4,TimeBand.PEAK,"2.90");
put(m,5,5,TimeBand.PEAK,"1.90");

        // Off-peak
        put(m,1,1,TimeBand.OFF_PEAK,"2.00");
put(m,1,2,TimeBand.OFF_PEAK,"2.70"); put(m,1,3,TimeBand.OFF_PEAK,"3.20");
put(m,1,4,TimeBand.OFF_PEAK,"3.70"); put(m,1,5,TimeBand.OFF_PEAK,"4.20");
        put(m,2,1,TimeBand.OFF_PEAK,"2.70");
put(m,2,2,TimeBand.OFF_PEAK,"1.90"); put(m,2,3,TimeBand.OFF_PEAK,"2.60");
put(m,2,4,TimeBand.OFF_PEAK,"3.20"); put(m,2,5,TimeBand.OFF_PEAK,"3.80");
        put(m,3,1,TimeBand.OFF_PEAK,"3.20");
put(m,3,2,TimeBand.OFF_PEAK,"2.60"); put(m,3,3,TimeBand.OFF_PEAK,"1.70");
put(m,3,4,TimeBand.OFF_PEAK,"2.50"); put(m,3,5,TimeBand.OFF_PEAK,"3.10");
        put(m,4,1,TimeBand.OFF_PEAK,"3.70");
put(m,4,2,TimeBand.OFF_PEAK,"3.20"); put(m,4,3,TimeBand.OFF_PEAK,"2.50");
put(m,4,4,TimeBand.OFF_PEAK,"1.60"); put(m,4,5,TimeBand.OFF_PEAK,"2.40");
        put(m,5,1,TimeBand.OFF_PEAK,"4.20");
put(m,5,2,TimeBand.OFF_PEAK,"3.80"); put(m,5,3,TimeBand.OFF_PEAK,"3.10");
put(m,5,4,TimeBand.OFF_PEAK,"2.40"); put(m,5,5,TimeBand.OFF_PEAK,"1.50");

        return Map.copyOf(m);
    }
```

```
    private static void put(Map<String,BigDecimal> m, int f, int t,
TimeBand b, String a) {
        m.put(key(f,t,b), money(a));
    }
}


//
==============================================================================
==
// Journey
//
==============================================================================
==
class Journey {
    private static int counter = 1;

    private int id;
    private String date;
    private int fromZone;
    private int toZone;
    private int zonesCrossed;

    private CityRideDataset.PassengerType passengerType;
    private CityRideDataset.TimeBand timeBand;

    private BigDecimal baseFare;
    private BigDecimal discountAmount;
    private BigDecimal discountedFare;
    private BigDecimal chargedFare;

    public Journey(String date, int fromZone, int toZone,
                   CityRideDataset.PassengerType passengerType,
                   CityRideDataset.TimeBand timeBand) {
        this.id = counter++;
        this.date = date;
        this.fromZone = fromZone;
        this.toZone = toZone;
        this.passengerType = passengerType;
        this.timeBand = timeBand;

        this.zonesCrossed = Math.abs(toZone - fromZone) + 1;

        this.baseFare = CityRideDataset.getBaseFare(fromZone, toZone,
timeBand);

        BigDecimal rate = CityRideDataset.DISCOUNT_RATE.get(passengerType);
        this.discountAmount = baseFare.multiply(rate).setScale(2,
RoundingMode.HALF_UP);

        this.discountedFare = baseFare.subtract(discountAmount).setScale(2,
RoundingMode.HALF_UP);

        // chargedFare is set by manager after cap check
        this.chargedFare = discountedFare;
    }

    public static void resetCounter() { counter = 1; }
```

```
    public int getId() { return id; }
    public String getDate() { return date; }
    public int getFromZone() { return fromZone; }
    public int getToZone() { return toZone; }
    public int getZonesCrossed() { return zonesCrossed; }
    public CityRideDataset.PassengerType getPassengerType() { return
passengerType; }
    public CityRideDataset.TimeBand getTimeBand() { return timeBand; }

    public BigDecimal getBaseFare() { return baseFare; }
    public BigDecimal getDiscountAmount() { return discountAmount; }
    public BigDecimal getDiscountedFare() { return discountedFare; }
    public BigDecimal getChargedFare() { return chargedFare; }

    public void setChargedFare(BigDecimal chargedFare) {
        this.chargedFare = chargedFare;
    }

    public boolean involvesZone(int zone) {
        return fromZone == zone || toZone == zone;
    }

    public void display() {
        System.out.println(
                "ID:" + id +
                        " Date:" + date +
                        " From:" + fromZone +
                        " To:" + toZone +
                        " Band:" + timeBand +
                        " Passenger:" + passengerType +
                        " Zones:" + zonesCrossed +
                        " Base:£" + baseFare +
                        " Disc:£" + discountedFare +
                        " Charged:£" + chargedFare
        );
    }
}


//
==========================================================================
==
//   CityRideManager
//
==========================================================================
==
class CityRideManager {
    private List<Journey> journeys = new ArrayList<>();

    // Active session
    private String sessionName;
    private CityRideDataset.PassengerType sessionPassengerType;

    // Running totals for caps
    private Map<CityRideDataset.PassengerType, BigDecimal> runningTotals =
new HashMap<>();

    public CityRideManager(String sessionName,
CityRideDataset.PassengerType sessionPassengerType) {
        this.sessionName = sessionName;
```

```
        this.sessionPassengerType = sessionPassengerType;
        resetTotalsOnly();
    }

    public String getSessionName() { return sessionName; }
    public CityRideDataset.PassengerType getSessionPassengerType() { return
sessionPassengerType; }

    public void setSessionPassenger(String name,
CityRideDataset.PassengerType type) {
        this.sessionName = name;
        this.sessionPassengerType = type;
    }

    private void resetTotalsOnly() {
        runningTotals.clear();
        CityRideDataset.PassengerType[] types =
CityRideDataset.PassengerType.values();
        for (int i = 0; i < types.length; i++) {
            runningTotals.put(types[i], BigDecimal.ZERO);
        }
    }

    // ADD JOURNEY
    public void addJourney(String date, int from, int to,
CityRideDataset.TimeBand band) {
        CityRideDataset.PassengerType type = sessionPassengerType;

        Journey j = new Journey(date, from, to, type, band);

        // cap logic
        BigDecimal cap = CityRideDataset.DAILY_CAP.get(type);
        BigDecimal current = runningTotals.get(type);
        BigDecimal discounted = j.getDiscountedFare();

        BigDecimal charged;
        if (current.compareTo(cap) >= 0) {
            charged = BigDecimal.ZERO;
        } else if (current.add(discounted).compareTo(cap) > 0) {
            charged = cap.subtract(current);
        } else {
            charged = discounted;
        }

        charged = charged.setScale(2, RoundingMode.HALF_UP);
        j.setChargedFare(charged);

        runningTotals.put(type, current.add(charged));
        journeys.add(j);

        System.out.println("Journey added! Charged: £" + charged);
    }

    //  LIST JOURNEYS
    public void listAllJourneys() {
        if (journeys.isEmpty()) {
            System.out.println("No journeys recorded.");
            return;
        }
        System.out.println("\n--- All Journeys ---");
```

```
        for (int i = 0; i < journeys.size(); i++) {
            journeys.get(i).display();
        }
    }

    //  FILTER JOURNEY
    public void filterByPassengerType(CityRideDataset.PassengerType type) {
        List<Journey> results = new ArrayList<>();
        for (int i = 0; i < journeys.size(); i++) {
            Journey j = journeys.get(i);
            if (j.getPassengerType() == type) results.add(j);
        }
        printFiltered(results, "Passenger type: " + type);
    }

    public void filterByTimeBand(CityRideDataset.TimeBand band) {
        List<Journey> results = new ArrayList<>();
        for (int i = 0; i < journeys.size(); i++) {
            Journey j = journeys.get(i);
            if (j.getTimeBand() == band) results.add(j);
        }
        printFiltered(results, "Time band: " + band);
    }

    public void filterByZone(int zone) {
        List<Journey> results = new ArrayList<>();
        for (int i = 0; i < journeys.size(); i++) {
            Journey j = journeys.get(i);
            if (j.involvesZone(zone)) results.add(j);
        }
        printFiltered(results, "Zone involved: " + zone);
    }

    public void filterByDate(String date) {
        List<Journey> results = new ArrayList<>();
        for (int i = 0; i < journeys.size(); i++) {
            Journey j = journeys.get(i);
            if (j.getDate().equalsIgnoreCase(date)) results.add(j);
        }
        printFiltered(results, "Date: " + date);
    }

    private void printFiltered(List<Journey> results, String label) {
        System.out.println("\n--- Filter Results (" + label + ") ---");
        if (results.isEmpty()) {
            System.out.println("No journeys found.");
            return;
        }
        for (int i = 0; i < results.size(); i++) {
            results.get(i).display();
        }
    }

    //  REMOVE JOURNEY
    public void removeJourney(int id) {
        Journey toRemove = null;
        for (int i = 0; i < journeys.size(); i++) {
            if (journeys.get(i).getId() == id) {
                toRemove = journeys.get(i);
                break;
```

```
            }
        }

        if (toRemove == null) {
            System.out.println("No journey with ID " + id);
            return;
        }

        journeys.remove(toRemove);

        rebuildAllCharges();

        System.out.println("Journey removed and totals recalculated.");
    }

    private void rebuildAllCharges() {
        resetTotalsOnly();

        // Recalculate chargedFare for every journey in the remaining list
        for (int i = 0; i < journeys.size(); i++) {
            Journey j = journeys.get(i);
            CityRideDataset.PassengerType type = j.getPassengerType();

            BigDecimal cap = CityRideDataset.DAILY_CAP.get(type);
            BigDecimal current = runningTotals.get(type);
            BigDecimal discounted = j.getDiscountedFare();

            BigDecimal charged;
            if (current.compareTo(cap) >= 0) {
                charged = BigDecimal.ZERO;
            } else if (current.add(discounted).compareTo(cap) > 0) {
                charged = cap.subtract(current);
            } else {
                charged = discounted;
            }

            charged = charged.setScale(2, RoundingMode.HALF_UP);
            j.setChargedFare(charged);
            runningTotals.put(type, current.add(charged));
        }
    }

    // RESET
    public void resetDay() {
        journeys.clear();
        resetTotalsOnly();
        Journey.resetCounter();
        System.out.println("Day reset.");
    }

    //  DAILY SUMMARY
    public void showDailySummary() {
        if (journeys.isEmpty()) {
            System.out.println("No journeys to summarise.");
            return;
        }

        BigDecimal total = BigDecimal.ZERO;
        Journey mostExpensive = journeys.get(0);
```

```
        for (int i = 0; i < journeys.size(); i++) {
            Journey j = journeys.get(i);
            total = total.add(j.getChargedFare());
            if
(j.getChargedFare().compareTo(mostExpensive.getChargedFare()) > 0) {
                mostExpensive = j;
            }
        }

        BigDecimal avg = total.divide(new BigDecimal(journeys.size()), 2,
RoundingMode.HALF_UP);

        System.out.println("\n--- Daily Summary ---");
        System.out.println("Total journeys: " + journeys.size());
        System.out.println("Total charged: £" + total);
        System.out.println("Average per journey: £" + avg);
        System.out.println("Most expensive journey: ID " +
mostExpensive.getId() + " (£" + mostExpensive.getChargedFare() + ")");
    }

    //  TOTALS BY PASSENGER TYPE
    public void showTotalsByPassengerType() {
        System.out.println("\n--- Totals By Passenger Type ---");

        CityRideDataset.PassengerType[] types =
CityRideDataset.PassengerType.values();
        for (int t = 0; t < types.length; t++) {
            CityRideDataset.PassengerType type = types[t];

            int count = 0;
            BigDecimal preDiscountSum = BigDecimal.ZERO;
            BigDecimal postDiscountSum = BigDecimal.ZERO;
            BigDecimal chargedSum = BigDecimal.ZERO;

            for (int i = 0; i < journeys.size(); i++) {
                Journey j = journeys.get(i);
                if (j.getPassengerType() == type) {
                    count++;
                    preDiscountSum = preDiscountSum.add(j.getBaseFare());
                    postDiscountSum =
postDiscountSum.add(j.getDiscountedFare());
                    chargedSum = chargedSum.add(j.getChargedFare());
                }
            }

            BigDecimal cap = CityRideDataset.DAILY_CAP.get(type);
            boolean capReached = runningTotals.get(type).compareTo(cap) >=
0;

            System.out.println("\n" + type);
            System.out.println("Journeys: " + count);
            System.out.println("Pre-discount total: £" +
preDiscountSum.setScale(2, RoundingMode.HALF_UP));
            System.out.println("Post-discount total: £" +
postDiscountSum.setScale(2, RoundingMode.HALF_UP));
            System.out.println("Charged total (after cap): £" +
chargedSum.setScale(2, RoundingMode.HALF_UP));
            System.out.println("Daily cap: £" + cap.setScale(2,
RoundingMode.HALF_UP));
            System.out.println("Cap reached: " + (capReached ? "YES" :
```

```
"NO"));
        }
    }

    //  CATEGORY COUNTS
    public void showCategoryCounts() {
        System.out.println("\n--- Category Counts ---");

        int peak = 0;
        int offPeak = 0;

        // Zone involvement - how many times each zone appears as from or
to
        int[] zoneInvolvement = new int[CityRideDataset.MAX_ZONE + 1]; //
index 1..5

        // Route pair counts
        Map<String, Integer> routeCounts = new LinkedHashMap<>();

        for (int i = 0; i < journeys.size(); i++) {
            Journey j = journeys.get(i);

            if (j.getTimeBand() == CityRideDataset.TimeBand.PEAK) peak++;
            else offPeak++;

            zoneInvolvement[j.getFromZone()]++;
            zoneInvolvement[j.getToZone()]++;

            String pair = j.getFromZone() + " -> " + j.getToZone();
            if (!routeCounts.containsKey(pair)) routeCounts.put(pair, 1);
            else routeCounts.put(pair, routeCounts.get(pair) + 1);
        }

        System.out.println("Peak journeys: " + peak);
        System.out.println("Off-peak journeys: " + offPeak);

        System.out.println("Zone involvement counts:");
        for (int z = CityRideDataset.MIN_ZONE; z <=
CityRideDataset.MAX_ZONE; z++) {
            System.out.println("Zone " + z + ": " + zoneInvolvement[z]);
        }

        System.out.println("Route (zone pair) counts:");
        if (routeCounts.isEmpty()) {
            System.out.println("None");
        } else {
            for (String key : routeCounts.keySet()) {
                System.out.println(key + " : " + routeCounts.get(key));
            }
        }
    }

    //  Helpers
    public boolean journeyExists(int id) {
        for (int i = 0; i < journeys.size(); i++) {
            if (journeys.get(i).getId() == id) return true;
        }
        return false;
    }
```

```
    public boolean hasJourneys() {
        return !journeys.isEmpty();
    }
}


//
==========================================================================
==
// InputHelper - validation + re-prompt
//
==========================================================================
==
class InputHelper {
    private Scanner scanner;

    public InputHelper(Scanner scanner) { this.scanner = scanner; }

    public String readNonBlank(String prompt) {
        while (true) {
            System.out.print(prompt);
            String s = scanner.nextLine().trim();
            if (!s.isEmpty()) return s;
            System.out.println("Input required.");
        }
    }


    public String readOptional(String prompt) {
        System.out.print(prompt);
        return scanner.nextLine().trim();
    }

    public int readInt(String prompt) {
        while (true) {
            try {
                System.out.print(prompt);
                String s = scanner.nextLine().trim();
                return Integer.parseInt(s);
            } catch (Exception e) {
                System.out.println("Invalid number.");
            }
        }
    }

    public int readZone(String prompt) {
        while (true) {
            int z = readInt(prompt);
            if (z >= 1 && z <= 5) return z;
            System.out.println("Zone must be 1-5.");
        }
    }

    public CityRideDataset.TimeBand readTimeBand(String prompt) {
        while (true) {
            System.out.print(prompt);
            String s = scanner.nextLine().trim().toLowerCase();

            if (s.equals("peak")) return CityRideDataset.TimeBand.PEAK;
            if (s.equals("off-peak") || s.equals("offpeak") ||
```

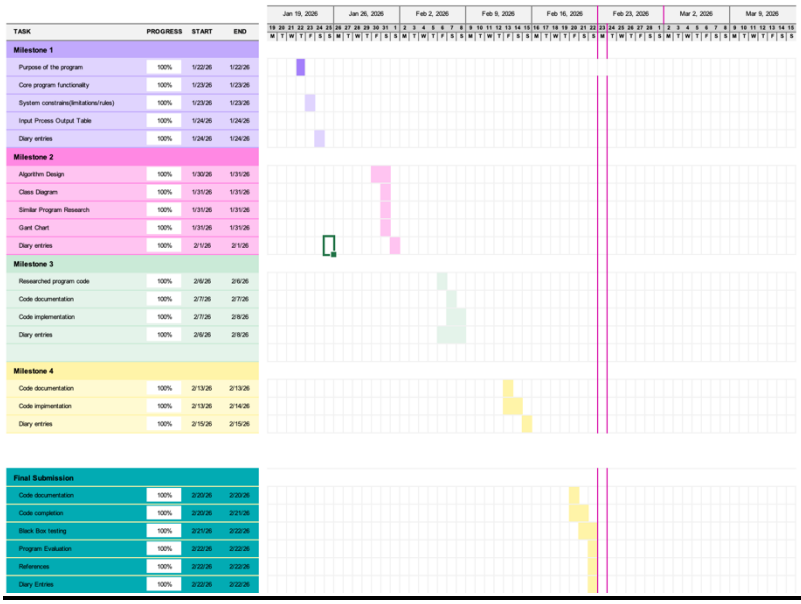
```
s.equals("off peak") || s.equals("off")) {
                return CityRideDataset.TimeBand.OFF_PEAK;
            }
            System.out.println("Enter Peak or Off-peak.");
        }
    }

    public CityRideDataset.PassengerType readPassengerTypeMenu() {
        System.out.println("Select passenger type:");
        System.out.println("1) Adult");
        System.out.println("2) Student");
        System.out.println("3) Child");
        System.out.println("4) Senior Citizen");
        int t;
        while (true) {
            t = readInt("Choice: ");
            if (t >= 1 && t <= 4) break;
            System.out.println("Choose 1-4.");
        }

        if (t == 1) return CityRideDataset.PassengerType.ADULT;
        if (t == 2) return CityRideDataset.PassengerType.STUDENT;
        if (t == 3) return CityRideDataset.PassengerType.CHILD;
        return CityRideDataset.PassengerType.SENIOR_CITIZEN;
    }

    public boolean readYesNo(String prompt) {
        while (true) {
            System.out.print(prompt + " (yes/no): ");
            String r = scanner.nextLine().trim().toLowerCase();
            if (r.equals("yes") || r.equals("y")) return true;
            if (r.equals("no") || r.equals("n")) return false;
            System.out.println("Type yes or no.");
        }
    }
}


//
==========================================================================
==
//   CityRideLiteApp - main menu
//
==========================================================================
==
public class CityRideLiteApp {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        InputHelper input = new InputHelper(sc);

        System.out.println("Welcome to CityRide Lite!");

        // Active passenger session - type + optional name
        String name = input.readOptional("Enter passenger name (optional,
press Enter to skip): ");
        CityRideDataset.PassengerType type = input.readPassengerTypeMenu();

        CityRideManager manager = new CityRideManager(name, type);
```

```
        System.out.println("Active passenger type: " +
manager.getSessionPassengerType() +
                (manager.getSessionName().isEmpty() ? "" : " | Name: " +
manager.getSessionName()));

        int choice;
        do {
            printMenu();
            choice = input.readInt("Enter choice: ");

            switch (choice) {
                case 1:
                    String date = input.readNonBlank("Date (dd/mm/yyyy):
");
                    int from = input.readZone("From Zone (1-5): ");
                    int to = input.readZone("To Zone (1-5): ");
                    CityRideDataset.TimeBand band =
input.readTimeBand("Time band (Peak/Off-peak): ");
                    manager.addJourney(date, from, to, band);
                    break;

                case 2:
                    manager.listAllJourneys();
                    break;

                case 3:
                    filterMenu(input, manager);
                    break;

                case 4:
                    manager.showDailySummary();
                    manager.showTotalsByPassengerType();
                    manager.showCategoryCounts();
                    break;

                case 5:
                    manager.showTotalsByPassengerType();
                    break;

                case 6:
                    int id = input.readInt("Journey ID to remove: ");
                    if (!manager.journeyExists(id)) {
                        System.out.println("No journey with ID " + id);
                    } else if (input.readYesNo("Remove journey " + id +
"?")) {
                        manager.removeJourney(id);
                    } else {
                        System.out.println("Cancelled.");
                    }
                    break;

                case 7:
                    if (input.readYesNo("Reset all journeys?"))
manager.resetDay();
                    else System.out.println("Cancelled.");
                    break;

                case 8:
                    // Switch active passenger session
                    String newName = input.readOptional("Enter passenger
```

```
name (optional, press Enter to skip): ");
                    CityRideDataset.PassengerType newType =
input.readPassengerTypeMenu();
                    manager.setSessionPassenger(newName, newType);
                    System.out.println("Active passenger changed to: " +
manager.getSessionPassengerType() +
                            (manager.getSessionName().isEmpty() ? "" : " |
Name: " + manager.getSessionName()));
                    break;

                case 9:
                    System.out.println("Goodbye!");
                    break;

                default:
                    System.out.println("Invalid choice.");
            }

        } while (choice != 9);

        sc.close();
    }

    private static void printMenu() {
        System.out.println("\n1) Add Journey");
        System.out.println("2) List All Journeys");
        System.out.println("3) Filter Journeys");
        System.out.println("4) Daily Summary (includes totals + category
counts)");
        System.out.println("5) Totals by Passenger Type");
        System.out.println("6) Remove Journey");
        System.out.println("7) Reset Day");
        System.out.println("8) Switch Active Passenger");
        System.out.println("9) Exit");
    }

    private static void filterMenu(InputHelper input, CityRideManager
manager) {
        System.out.println("\nFilter by:");
        System.out.println("1) Passenger Type");
        System.out.println("2) Time Band");
        System.out.println("3) Zone involved");
        System.out.println("4) Date");
        int fc = input.readInt("Choice: ");

        switch (fc) {
            case 1:
                CityRideDataset.PassengerType pt =
input.readPassengerTypeMenu();
                manager.filterByPassengerType(pt);
                break;

            case 2:
                CityRideDataset.TimeBand band = input.readTimeBand("Time
band (Peak/Off-peak): ");
                manager.filterByTimeBand(band);
                break;

            case 3:
                int zone = input.readZone("Zone (1-5): ");
```

```
                manager.filterByZone(zone);
                break;

            case 4:
                String date = input.readNonBlank("Date (dd/mm/yyyy): ");
                manager.filterByDate(date);
                break;

            default:
                System.out.println("Invalid filter choice.");
        }
    }
}
```

Gantt Chart



TASK	PROGRESS	START	END
Milestone 1			
Purpose of the program	100%	1/22/26	1/22/26
Core program functionality	100%	1/23/26	1/23/26
System constraints(limitations/rules)	100%	1/23/26	1/23/26
Input Process Output Table	100%	1/24/26	1/24/26
Diary entries	100%	1/24/26	1/24/26
Milestone 2			
Algorithm Design	100%	1/30/26	1/31/26
Class Diagram	100%	1/31/26	1/31/26
Similar Program Research	100%	1/31/26	1/31/26
Gant Chart	100%	1/31/26	1/31/26
Diary entries	100%	2/1/26	2/1/26
Milestone 3			
Researched program code	100%	2/6/26	2/6/26
Code documentation	100%	2/7/26	2/7/26
Code implementation	100%	2/7/26	2/8/26
Diary entries	100%	2/8/26	2/8/26
Milestone 4			
Code documentation	100%	2/13/26	2/13/26
Code implmentation	100%	2/13/26	2/14/26
Diary entries	100%	2/15/26	2/15/26
Final Submission			
Code documentation	100%	2/20/26	2/20/26
Code completion	100%	2/20/26	2/21/26
Black Box testing	100%	2/21/26	2/22/26
Program Evaluation	100%	2/22/26	2/22/26
References	100%	2/22/26	2/22/26
Diary Entries	100%	2/22/26	2/22/26

References

Oracle. (2024). Class Scanner. Retrieved from: https://docs.oracle.com/en/java/javase/ [Accessed 22 February 2026].

Oracle. (2019). The Java™ Tutorials: Enum Types. Retrieved from: https://docs.oracle.com/javase/tutorial/java/javaOO/enum.html [Accessed 22 February 2026].

Oracle. (2019). The switch Statement (Java Tutorials). Retrieved from: https://docs.oracle.com/javase/tutorial/java/nutsandbolts/switch.html [Accessed 22 February 2026].

Oracle. (2019). Collections Framework Overview. Retrieved from: https://docs.oracle.com/javase/tutorial/collections/[Accessed 22 February 2026].

Stack Overflow. (2026). Questions and answers on Java input validation and exception handling. Retrieved from: https://stackoverflow.com/ [Accessed 22 February 2026].

W3Schools. (2025). Java Tutorial. Retrieved from: https://www.w3schools.com/java/ [Accessed 22 February 2026].

GeeksforGeeks. (2024). LinkedHashMap in Java. Retrieved from: https://www.geeksforgeeks.org/linkedhashmap-class-in-java/ [Accessed 22 February 2026].

GeeksforGeeks. (2024). Exception Handling in Java. Retrieved from: https://www.geeksforgeeks.org/exceptions-in-java/ [Accessed 22 February 2026].

OpenAI. (2025). ChatGPT. Retrieved from: https://chat.openai.com/ [Accessed 22 February 2026].

Appendices

IY4113 Milestone 1

Purpose of the Program

To simulate a public transport system in a console-based Java application which allows a user to add as many journeys by prompting the user to enter the date, starting zone, passenger type and if the trip was peak or off peak. After it calculates the zones crossed and correct fare. It also allows the user to remove a journey if a mistake is made or reset whole day after confirmation. Lastly, it helps the rider keep track of fares, monitor expenses and understand how caps and discounts affect the total cost of their journeys.

Input Process Output Table

![ ](/Users/macbook/Desktop/Screenshot%202026-01-24%20at%2019.29.15.png)IPO table (It maybe easier to create the table in Word and paste as an image!)*

Gantt Chart

Diary Entries

22/01/2026 – Diary Entry 1: Understanding the CityRide Lite Requirements & Initial Planning

Today I focused on understanding the CityRide Lite project requirements in detail. I carefully read through the assessment brief and requirements so i can gain a clear picture of what the program is expected to do. I broke down what is needed into user input, journey storage, fare calculation, discounts, daily caps, summaries, and error handling. I did this to make sure I fully understood system.

After going through the requirements, I found the main components of the program, including adding journeys, listing and filtering journeys, calculating fares using the provided details, and producing daily summaries and totals by passenger type. This helped me understand how the program would operate.

One difficulty I encountered was the large number of functional requirements and deciding which ones represented core user-facing features and internal calculations. I solved this by focusing on what the user selects from the menu as the main features, and treating zone calculations and fare

lookups as background processing within those actions. This made things clearer and helped prepare for the IPO tables and Gantt chart.

23/01/2026 – Diary Entry 2: Planning the Gantt Chart and Structuring the Analysis

Today I worked on planning the project timeline and creating the Gantt chart. I listed all the major activities required to complete the CityRide Lite project which I organised across weekly time periods to show a logical progression from understanding the problem to final submission.

I structured the timeline so that the early stages focused on analysis, IPO tables, and system design, before moving on to implementation and testing in later weeks. I did this because a clear design is essential before starting to code, especially for a program with complex rules such as fare calculations, discounts, and daily caps.

A challenge I faced was deciding how detailed the Gantt chart tasks should be. Initially, some tasks were too broad, such as “Design program,” which did not clearly show progress. I improved this by breaking tasks into more specific activities, such as “Design journey data structure,” “Plan fare calculation logic,” and “Design menu flow.” This made the Gantt chart more realistic and easier to understand.

24/01/2026 – Diary Entry 3: Finalising IPO Tables and Reviewing the Plan

Today I focused on finalising the analysis documentation, particularly the IPO tables and overall project plan. I created IPO tables for the main CityRide Lite features, including adding journeys, listing and filtering journeys, viewing daily summaries, viewing totals by passenger type, removing journeys, and resetting the day. This helped show how inputs, processes, and outputs flow through the system.

I checked the functional requirements to ensure that each required feature was represented.

One issue I noticed was that some features were doing the same functions, which initially made the analysis feel repetitive. To fix this, I changed the wording of the IPO tables to differentiate between user actions and internal processing. By the end of it, I had a clear, complete analysis and planning document that shows the CityRide Lite requirements and provides a strong foundation for the implementation phase.

Milestone 2

Algorithm Design

- *Add **images** for the design of your algorithm. Choose either Flowchart or JSP diagrams to demonstrate the functional elements of the algorithm. There should be multiple images for this part as you are decomposing the problem into smaller elements.*

- *Include a class diagram to demonstrate the class structure of the proposed program design.*

Research (minimum of 1 required, preferably 2)

Research existing programs that solve a similar problem. The program does not have to be written in java or object orientated in nature - just solve a similar type of problem.

Use the strucutre below to capture your evidence:

----------Name of program: Farebot

Reference (link): https://play.google.com/store/apps/details?id=com.codebutler.farebot

What it does well (2-3 features that work effectively):

The program is able to calculate and display the total amount of money spent on travel over a certain period of time which helps users see the overall travel costs

The program is able to show the user their journey history providing dates, routes and total costs

What it does poorly (at least 1 feature):

The program is quite outdated and not that easy to use as compared to modern transport apps

Key design ideas you could reuse (e.g., layout, navigation, input/output, program structure):

Storing journeys in a list with timestamps and costs which is similar to the journey list in the CityRide App

The program provides a summary of daily or session based travel expenses

Screenshot (showing the interface/output):

Name of program: Transit App

Reference (link): [Transit - the best app for buses and trains](https://transitapp.com/?utm_source=chatgpt.com)

What it does well (2-3 features that work effectively):

The program is able to display real time arrivals for both buses and trains

The program helps users to plan specific journeys with step by step directions until the destination

The program has a simple interface which provides quick and easy journey decisions

What it does poorly (at least 1 feature):

The program does not have detailed fare calculations and summaries of the totals

Key design ideas you could reuse (e.g., layout, navigation, input/output, program structure):

The program provides feedback after each user input making it interactive which is needed in the CityRide app

Screenshot (showing the interface/output):

Updated Gantt Chart

Add your updated Gantt chart here!

Diary Entries

Friday – Researching Similar Programs

Date: Friday, 30 January 2026

What I did:

I researched existing programs that solve a similar problem to the CityRide app system. I focused on finding an application that tracks journeys and calculates travel costs. I chose Farebot and Transit App which i got of a google search on apps that help users access transport. This were my main examples and reviewed information from its Google Play Store page. I looked at how they both keep journey history, display routes and dates, and calculate total travel costs over a period of time.

Why I did it:

This research was carried out to understand how real world systems manage travel fare data and present it to users. By analysing Farebot and Transit App, I was able to see how journey information can be stored in a structured way and how summaries can help users understand their spending. This helped me identify useful features that could be adapted for my own project such as storing journeys in a list and showing daily or session based totals.

Problems encountered:

One problem was that Farebot is quite outdated, meaning its interface is not very modern. This made it harder to compare it directly with newer applications. However, this also helped me focus more on functionality rather than design, which is more relevant to my coursework.

On the other hand the Transit App was quite modern and is the better one in this case

---

Saturday – Analysing Design Ideas and Updating Plans

Date: Saturday, 31 February 2026

What I did:

I looked at Farebot and Transit App in detail and found key design ideas that could be reused in the CityRide app. I focused on how journeys are stored with timestamps and costs, and how the app provides summaries of travel expenses. I then documented what they do well and what they do poorly linking these observations to how I plan to design the CityRide system. I also began reviewing my project plan and checked which tasks had been completed so far.

Why I did it:

This step was important to ensure that my project design is informed by existing solutions. By evaluating both the strengths and weaknesses of the apps, I could make decisions about what features to include and what to avoid. Reviewing my progress also helped me prepare to update my Gantt chart.

Problems encountered:

It was sometimes difficult to decide which features were relevant to my project and which were outside the scope of the coursework. I resolved this by focusing only on features related to journey tracking, fare calculation, and summary outputs, which directly match the CityRide requirements.

Sunday – Updating the Gantt Chart and Reflecting on Progress

Date: Sunday, 1 February 2026

What I did:

I updated my Gantt chart to show the current status of the project. This included marking the research phase as complete, confirming that the algorithm design using flowcharts had been finished and adjusting future tasks such as implementation and testing. I reviewed the entire project time to ensure it has clearly showed progress over time.

Why I did it:

Updating the Gantt chart was necessary to demonstrate good project management and organisation. It helps show that enough time is given for each part. This also allowed me to look at how the research and planning stages support the later development of the program.

Problems encountered:

One issue was deciding how much time to allocate to each remaining stage of the project. I fixed this by checking the complexity of each task and allowing extra time for testing and debugging, which are often more time consuming than expected.

Milestone 3

Research (minimum of 2, at least 3)

Conduct research to support your coding process, including use of code examples, tutortials, documentation and AI tools (if used).

Use the structure below to capture your evidence:

Title of research: Using ArrayList to Store Objects in Java

Reference (link): [Java ArrayList](https://www.w3schools.com/java/java_arraylist.asp)

How does the research help with coding practise?

This research was on understanding how the ArrayList class can be used to store multiple objects during program execution. Since the CityRide Lite application requires all journeys to be stored in memory for a single session, a data structure was required.

The W3Schools tutorial provided explanations and examples of how to create an ArrayList, add objects using the add() method, retrieve elements and iterate through them using a for each loop. This helped the implementation of the journeyList within the CityRideManager class where each Journey object is stored after being created.

Understanding how collections work helped my ability to manage dynamic data without using arrays of fixed size. It also ensured that journeys could be displayed, counted, and processed when generating summaries.

Key coding ideas you could reuse in your program:

Declaring ArrayList<Journey>

Using add() to insert objects

Using size() to count stored items

Using for-loops to display stored journeys

Checking if a list is empty before printing results

Screenshot of research:

Title of research:

Using ChatGPT to check code structure and program design

Reference (link):

https://chat.openai.com/

How does the research help with coding practise?:

ChatGPT was used as a research and review tool during the process of the CityRide Lite application. Rather than making a solution, it was used to review program structure and suggest improvements in clarity, organisation, and logical flow.

For example, at early implementation, i was not sure whether to place all logic in the main class or separate it into multiple classes. ChatGPT suggested maintaining a similar design pattern to the classroom example (Car and Garage), where one class represents the data model (Journey) and another manages operations (CityRidemManager). This helped the separation of responsibilities and improved readability.

ChatGPT was also used to confirm the logic for daily cap enforcement and discount calculations. It helped ensure that the charged fare would not exceed the passenger type, daily limit.

It helping improve my confidence in decision-making and professional programming practices.

Key coding ideas you could reuse in your program:

Structuring the program into separate classes (Journey, Manager, Main)

Implementing daily cap logic using conditional statements

Ensuring consistent naming

Improving menu flow clarity

Refining conditional structures for discounts and totals

Checking for logical errors in fare calculation

Screenshot of research:

Title of research: Creating Menu-Driven Programs in Java Using Loops and Switch Statements

Reference (link):[Java Switch](https://www.w3schools.com/java/java_switch.asp)

How does the research help with coding practise?

This research is based on how to build a repeating console menu using loops and conditional structures. The CityRide Lite program needs the main menu to display until the user selects the exit option. The W3Schools tutorials on while loops and switch statements provided practical examples of how user input can control program flow.

This supported the implementation of the do-while loop in the main class which ensures that the menu remains active throughout the session. The switch structure also allows different program features (add journey, view journeys, summary) to be executed based on user selection.

Key coding ideas you could reuse in your program:

Using do-while to repeat the menu

Using switch for user choices

Capturing input using Scanner

Preventing program termination on invalid menu options

Structuring clear user prompts

Screenshot of research:

Title of research:

Understanding Object-Oriented Programming and Encapsulation in Java

Reference (link):

[Java OOP (Object-Oriented Programming)](https://www.w3schools.com/java/java_oop.asp)

[Java Encapsulation and Getters and Setters](https://www.w3schools.com/java/java_encapsulation.asp)

How does the research help with coding practise?:

This focused on object-oriented programming principles, particularly encapsulation and class structure. The CityRide Lite application follows a similar design pattern to the Car and Garage example which we did in class where one class represents the data model (Journey) and another class manages the operations (CityRide Manager)

The W3Schools OOP tutorials shows how private variables protect object data and how getters allow controlled access. This helped the design of the Journey class ensuring that attributes such as id, fromZone and chargedFare remain private and are accessed only through methods.

Separating responsibilities between classes improved program organisation and readability. The main class handles user interaction while the manager class handles business logic like adding journeys and calculating fares. This shows following of the professional programming standards and clear structural design.

Key coding ideas you could reuse in your program:

Creating classes with private attributes

Using constructors to initialise objects

Implementing getter methods

Separating data and management logic

Following structured OOP design

Screenshot of research:

Program Code

Paste the current program code created so far. It does not have to be runnable code (document though if it does not work!)

Program code goes here:

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

class Journey {

    private int id;
    private int fromZone;
    private int toZone;
    private String passengerType;
    private String timeBand;
    private double chargedFare;

    public Journey(int id, int fromZone, int toZone,
                   String passengerType, String timeBand,
```

```
            double chargedFare) {

    this.id = id;
    this.fromZone = fromZone;
    this.toZone = toZone;
    this.passengerType = passengerType;
    this.timeBand = timeBand;
    this.chargedFare = chargedFare;
}

public int getId() {
    return id;
}

public String getPassengerType() {
    return passengerType;
}

public double getChargedFare() {
    return chargedFare;
}

public void displayDetails() {
    System.out.println("ID: " + id +
            " | From: " + fromZone +
            " | To: " + toZone +
            " | Passenger: " + passengerType +
            " | Time: " + timeBand +
            " | Charged: ¬£" + chargedFare);
}
```

```
}

class CityRideManager {

    private List<Journey> journeyList = new ArrayList<>();
    private int nextId = 1;

    private double adultTotal = 0;
    private double studentTotal = 0;
    private double childTotal = 0;
    private double seniorTotal = 0;

    public void addJourney(int fromZone, int toZone,
                           String passengerType, String timeBand) {

        int zonesCrossed = Math.abs(toZone - fromZone) + 1;

        double baseFare = zonesCrossed * 2.0;

        double discount = 0;
        double cap = 0;
        double currentTotal = 0;

        if (passengerType.equalsIgnoreCase("Adult")) {
            discount = 0;
            cap = 8.0;
            currentTotal = adultTotal;
        }
        else if (passengerType.equalsIgnoreCase("Student")) {
```

```
    discount = 0.25;
    cap = 6.0;
    currentTotal = studentTotal;
}
else if (passengerType.equalsIgnoreCase("Child")) {
    discount = 0.5;
    cap = 4.0;
    currentTotal = childTotal;
}
else if (passengerType.equalsIgnoreCase("Senior")) {
    discount = 0.3;
    cap = 7.0;
    currentTotal = seniorTotal;
}

double discountedFare = baseFare - (baseFare * discount);

double chargedFare;

if (currentTotal >= cap) {
    chargedFare = 0;
}
else if (currentTotal + discountedFare > cap) {
    chargedFare = cap - currentTotal;
}
else {
    chargedFare = discountedFare;
}

if (passengerType.equalsIgnoreCase("Adult")) {
```

```
        adultTotal += chargedFare;
    }
    else if (passengerType.equalsIgnoreCase("Student")) {
        studentTotal += chargedFare;
    }
    else if (passengerType.equalsIgnoreCase("Child")) {
        childTotal += chargedFare;
    }
    else if (passengerType.equalsIgnoreCase("Senior")) {
        seniorTotal += chargedFare;
    }

    Journey j = new Journey(nextId++, fromZone, toZone,
            passengerType, timeBand, chargedFare);

    journeyList.add(j);

    System.out.println("Journey added. Charged ¬£" + chargedFare);
}

public void showAllJourneys() {
    if (journeyList.isEmpty()) {
        System.out.println("No journeys recorded.");
    } else {
        for (Journey j : journeyList) {
            j.displayDetails();
        }
    }
}
```

```
    public void showSummary() {

        int totalJourneys = journeyList.size();
        double totalCost = adultTotal + studentTotal + childTotal + seniorTotal;

        double average = 0;
        if (totalJourneys > 0) {
            average = totalCost / totalJourneys;
        }

        System.out.println("Total Journeys: " + totalJourneys);
        System.out.println("Total Cost: ¬£" + totalCost);
        System.out.println("Average Cost: ¬£" + average);
    }

}

public class CityRideLiteApp {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
        CityRideManager manager = new CityRideManager();

        int choice;

        do {
            System.out.println("\nCityRide Lite");
            System.out.println("1. Add Journey");
            System.out.println("2. View All Journeys");
```

```
System.out.println("3. Daily Summary");
System.out.println("4. Exit");
System.out.print("Choice: ");

choice = scanner.nextInt();
scanner.nextLine();

switch (choice) {

  case 1:
    System.out.print("Enter From Zone (1-5): ");
    int from = scanner.nextInt();

    System.out.print("Enter To Zone (1-5): ");
    int to = scanner.nextInt();
    scanner.nextLine();

    System.out.print("Enter Passenger Type (Adult/Student/Child/Senior): ");
    String passenger = scanner.nextLine();

    System.out.print("Enter Time Band (Peak/OffPeak): ");
    String time = scanner.nextLine();

    manager.addJourney(from, to, passenger, time);
    break;

  case 2:
    manager.showAllJourneys();
    break;
```

```
            case 3:
                manager.showSummary();
                break;

            case 4:
                System.out.println("Exiting...");
                break;

            default:
                System.out.println("Invalid choice.");
        }

    } while (choice != 4);

    scanner.close();
  }

}
```

Updated Gantt Chart

Add your updated Gantt chart here!

Diary Entries

Add diary entries here detailing what you have done, wny you have done it, and any problems encountered.

Thursday

Today I looked at the full assessment brief for CityRide Lite to ensure I clearly understood all functional requirements. I looked at fare rules, daily caps, passenger types and validation requirements. I updated my Gantt chart to show the progress.

I began reviewing my existing class structure from previous practical sessions (CarApp example) to make sure I would follow a similar structure for consistency. I found out that I would need a Journey class and a main application class to manage the menu and logic.

One difficulty i encountered today was understanding how the daily cap should behave when the total is close to the limit. I noted this as something needing further research.

Friday

Today I focused on research to support my implementation. I reviewed W3Schools documentation on ArrayList and try-catch exception handling to ensure I could correctly store journeys and validate user input. This helped me confirm that using an ArrayList would allow me to add, list and iterate through journeys.

I also used ChatGPT to review my cap logic structure. This helped me understand how to structure conditional statements so that the total never exceeds the daily cap and becomes £0.00 once the cap is reached.

I then implemented the basic program structure including:

Main menu loop

Add journey function

Basic fare calculation

Discount application

A challenge I faced was ensuring the running totals were updated correctly for each passenger type without mixing them.

Saturday

Today I completed the core functionality of the program, I implemented:

Daily cap enforcement

Running totals per passenger type

Summary calculations (total journeys, total cost, average cost)

Input validation for zones and passenger types

I tested multiple scenarios to confirm that:

The cap is never exceeded

Once the cap is reached, further journeys are charged £0.00

Invalid inputs do not crash the program

During testing, I found that incorrect input types (such as letters instead of numbers) caused errors. I resolved this using try-catch blocks based on my research.

I updated my Gantt chart to show the progress

Sunday

Today I finalised the program and tested all that i had included. I verified that the menu loops correctly until exit, summaries calculate accurately and totals reset when required.

I checked my research section and ensured it explained how external sources influenced my coding decisions particularly in relation to ArrayList usage, exception handling and cap logic implementation.

I looked at the code formatting and ensured comments were clear and brief. I confirmed that the program aligns with the functional requirements.

The project was completed today and all required things like research Gantt chart, diary entries and program code are now prepared for submission.

Milestone 4

Program Code

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;
class Journey {
private int id, fromZone, toZone, zonesCrossed;
private String date;
private CityRideDataset.PassengerType passengerType;
private CityRideDataset.TimeBand timeBand;
private BigDecimal baseFare, discountedFare, chargedFare;
public Journey(int id, String date, int fromZone, int toZone,
CityRideDataset.PassengerType passengerType,
CityRideDataset.TimeBand timeBand,
int zonesCrossed,
BigDecimal baseFare,
BigDecimal discountedFare,
BigDecimal chargedFare) {
this.id = id; this.date = date; this.fromZone = fromZone; this.toZone = toZone;
this.passengerType = passengerType; this.timeBand = timeBand; this.zonesCrossed =
zonesCrossed;
this.baseFare = baseFare; this.discountedFare = discountedFare; this.chargedFare =
chargedFare;
}
public int getId() { return id; }
public String getDate() { return date; }
public int getFromZone() { return fromZone; }
public int getToZone() { return toZone; }
public CityRideDataset.PassengerType getPassengerType() { return passengerType; }
public CityRideDataset.TimeBand getTimeBand() { return timeBand; }
```

```
public BigDecimal getChargedFare() { return chargedFare; }
public void display() {
System.out.println("ID:" + id + " Date:" + date + " From:" + fromZone + " To:" + toZone +
" Band:" + timeBand + " Passenger:" + passengerType +
" Zones:" + zonesCrossed + " Base:¬£" + baseFare +
" Disc:¬£" + discountedFare + " Charged:¬£" + chargedFare);
}
}
class JourneyManager {
private String sessionName;
private CityRideDataset.PassengerType sessionPassengerType;
private List<Journey> journeys = new ArrayList<>();
private int nextId = 1;
private BigDecimal adultTotal = BigDecimal.ZERO, studentTotal = BigDecimal.ZERO,
childTotal = BigDecimal.ZERO, seniorTotal = BigDecimal.ZERO;
public JourneyManager(String sessionName, CityRideDataset.PassengerType type) {
this.sessionName = sessionName; this.sessionPassengerType = type;
}
public String getSessionName() { return sessionName; }
public CityRideDataset.PassengerType getSessionPassengerType() { return
sessionPassengerType; }
private BigDecimal getTotal(CityRideDataset.PassengerType t) {
switch(t) {
case ADULT: return adultTotal; case STUDENT: return studentTotal;
case CHILD: return childTotal; default: return seniorTotal;
}
}
private void updateTotal(CityRideDataset.PassengerType t, BigDecimal amt) {
switch(t) {
case ADULT: adultTotal = adultTotal.add(amt); break;
```

```
case STUDENT: studentTotal = studentTotal.add(amt); break;
case CHILD: childTotal = childTotal.add(amt); break;
default: seniorTotal = seniorTotal.add(amt);
}
}
public void addJourney(String date, int from, int to, CityRideDataset.PassengerType type,
CityRideDataset.TimeBand band) {
int zones = Math.abs(to - from) + 1;
BigDecimal base = CityRideDataset.getBaseFare(from, to, band);
BigDecimal discount =
base.multiply(CityRideDataset.DISCOUNT_RATE.get(type)).setScale(2,RoundingMode.HALF
_UP);
BigDecimal discounted = base.subtract(discount).setScale(2,RoundingMode.HALF_UP);
BigDecimal cap = CityRideDataset.DAILY_CAP.get(type);
BigDecimal running = getTotal(type);
BigDecimal charged = running.compareTo(cap) >=0 ? BigDecimal.ZERO :
running.add(discounted).compareTo(cap) >0 ? cap.subtract(running) :
discounted;
updateTotal(type, charged);
journeys.add(new Journey(nextId++, date, from, to, type, band, zones, base, discounted,
charged));
System.out.println("\nJourney added! Charged: ¬£" + charged);
}
public void listAllJourneys() {
if(journeys.isEmpty()) { System.out.println("No journeys recorded."); return; }
System.out.println("\n--- All Journeys ---");
for(Journey j : journeys) j.display();
}
public void removeJourney(int id) {
Journey toRemove = null;
```

```
for(Journey j: journeys) if(j.getId()==id) toRemove=j;
if(toRemove==null) { System.out.println("No journey with ID "+id); return; }
journeys.remove(toRemove);
recalcTotals();
System.out.println("Journey removed and totals recalculated.");
}
private void recalcTotals() {
adultTotal=studentTotal=childTotal=seniorTotal=BigDecimal.ZERO;
List<Journey> oldJourneys = new ArrayList<>(journeys);
journeys.clear(); nextId=1;
for(Journey j: oldJourneys) addJourney(j.getDate(), j.getFromZone(), j.getToZone(),
j.getPassengerType(), j.getTimeBand());
}
public void resetDay() { journeys.clear();
adultTotal=studentTotal=childTotal=seniorTotal=BigDecimal.ZERO; nextId=1;
System.out.println("Day reset."); }
public void showDailySummary() {
if(journeys.isEmpty()) { System.out.println("No journeys to summarise."); return; }
BigDecimal total = adultTotal.add(studentTotal).add(childTotal).add(seniorTotal);
BigDecimal avg = total.divide(new
BigDecimal(journeys.size()),2,RoundingMode.HALF_UP);
System.out.println("\n--- Daily Summary --- Total journeys: " + journeys.size() + " Total: £"
+ total + " Average: £" + avg);
}
}
class InputHelper {
private Scanner scanner;
public InputHelper(Scanner s) { scanner=s; }
public int readInt(String prompt){ while(true){ try{ System.out.print(prompt); return
Integer.parseInt(scanner.nextLine().trim()); } catch(Exception e){ System.out.println("Invalid
```

```
number"); } } }
public int readZone(String prompt){ int z; while(true){ z=readInt(prompt); if(z>=1 && z<=5)
return z; System.out.println("Zone must be 1-5."); } }
public String readDate(String prompt){ String d; while(true){ System.out.print(prompt);
d=scanner.nextLine().trim(); if(!d.isEmpty()) return d; System.out.println("Date required."); } }
public boolean readYesNo(String prompt){ String r; while(true){ System.out.print(prompt+"
(yes/no): "); r=scanner.nextLine().trim().toLowerCase(); if(r.equals("yes")||r.equals("y")) return
true; if(r.equals("no")||r.equals("n")) return false; System.out.println("Type yes or no."); } }
}
public class CityRideLiteApp {
public static void main(String[] args){
Scanner sc = new Scanner(System.in); InputHelper input=new InputHelper(sc);
System.out.println("Welcome to CityRide Lite!");
String name=input.readDate("Enter your name (optional, press Enter to skip): ");
CityRideDataset.PassengerType type=CityRideDataset.PassengerType.ADULT;
System.out.println("Select passenger type (1-Adult,2-Student,3-Child,4-Senior):");
int t=input.readInt("Choice: "); if(t==2) type=CityRideDataset.PassengerType.STUDENT;
else if(t==3) type=CityRideDataset.PassengerType.CHILD; else if(t==4)
type=CityRideDataset.PassengerType.SENIOR_CITIZEN;
JourneyManager manager=new JourneyManager(name,type);
int choice;
do{
System.out.println("\n1.Add Journey 2.List All 3.Remove 4.Summary 5.Reset 6.Exit");
choice=input.readInt("Enter choice: ");
switch(choice){
case 1:
String date=input.readDate("Date (dd/mm/yyyy): ");
int f=input.readZone("From Zone: ");
int to=input.readZone("To Zone: ");
manager.addJourney(date,f,to,type,CityRideDataset.TimeBand.PEAK);
```

```
break;
case 2: manager.listAllJourneys(); break;
case 3:
int id=input.readInt("Journey ID to remove: ");
if(input.readYesNo("Remove journey "+id+"?")) manager.removeJourney(id); else
System.out.println("Cancelled.");
break;
case 4: manager.showDailySummary(); break;
case 5:
if(input.readYesNo("Reset all journeys?")) manager.resetDay(); else
System.out.println("Cancelled.");
break;
}
} while(choice!=6);
System.out.println("Goodbye!"); sc.close();
}
}
```

Updated Gantt Chart

Diary Entries

Thursday 12th February 2026

I started the day by reviewing all the in class code we had covered so far. This included the original CityRide Lite program with the Journey and CityRideManager classes as well as the Student grading program. I tried understanding how each class interacted, how fares were calculated for different passenger types and how daily caps were applied. I also traced the logic for applying discounts to ensure I fully understood the calculations. Some parts, like the total fare recalculation after removing a journey were a bit tricky to follow so I made note of it to

refer back to them later.

Friday 13th February 2026

I continued by checking the Student grading code. I reviewed how the GradingRules class worked with static constants for grade thresholds and pass marks. I practiced converting numeric marks to letter grades using the GradeCalculator class and printing detailed student results including grade, pass/fail status and classification. I tested the code with sample data to confirm it was working correctly. This helped me understand the pattern of separating logic into classes and methods which I knew would be useful when extending the CityRide Lite program.

Saturday 14th February 2026

I focused on the updated CityRideDataset class which used enums and maps for fares, discounts, and daily caps. I looked at how the getBaseFare method worked with different zones and time bands. After that I started thinking about how to combine the original CityRide Lite code with this new dataset structure. I made a plan for one single program file that could use all these, I found which methods from the previous code needed updating like adding journeys, calculating discounted fares, and keeping running totals per passenger type.

Sunday 15th February 2026

Today I implemented the final version of the combined program. I created one file with all classes, Journey, JourneyManager, InputHelper and CityRideLiteApp. I added the CityRideDataset constants and methods making sure all fare calculations, discounts and daily caps worked correctly. I also included input validation for zones, dates, passenger type and time bands so that the program wouldn’t crash with invalid input. After checking adding journeys, listing journeys, filtering by passenger type or zone, removing journeys and resetting the day, everything ran as expected. The program now shows the functionality required by the class exercises and I documented each step in code comments for clarity.